Implementation of a Low Power IC for Neuromorphic Computing

ESE 441 Senior Design Progress Report

Spencer Wu (EE)
Arhaam Hossain (EE)
Huabin Wu (CE)
Ryan Lin (CE)

Faculty Advisor: Milutin Stanacevic

College of Engineering and Applied Sciences
Electrical & Computer Engineering Department
Stony Brook University

I. Abstract

This report summarizes the progress of development for a Spiking Neural Network (SNN) encoding framework optimized for low-power, ReRAM-based neuromorphic integrated circuits. While the initial objective was a native SNN, the inherent complexities of training from scratch necessitated a shift toward SpikingJelly's ANN-to-SNN (ANN2SNN) conversion pipeline to ensure project viability. As such, the design has pivoted from Time-to-First-Spike (TTFS) encoding with Leaky Integrate-and-Fire (LIF) neurons to rate coding utilizing Integrate-and-Fire (IF) neurons. This transition provides a more accessible implementation path while ensuring robust threshold calibration, hardware compatibility, and superior power efficiency.

The framework utilizes a ResNet architecture optimized via 8-bit quantization for both weights and input data. The software conversion has been finalized, and current efforts are now focused on the hardware mapping phase, translating neural layers onto a ReRAM crossbar utilizing differential pairs. In this layout, synaptic weights, initially defined as conductances ranging from 1 µS to 40 µS, are mapped to discrete memristive states, specifically a Low Resistance State (LRS) of 10 kΩ and a High Resistance State (HRS) of 300 kΩ. This is expected to take up 3 (256×256) cores to fully implement.

The input stage of the circuit is expected to process spectrogram data through a specialized two-phase 8-bit MAC protocol. Under this design, 8-bit pixels are split into 4-bit Most Significant Bit (MSB) and 4-bit Least Significant Bit (LSB) segments. These segments undergo integration to produce a digital Matrix-Vector Multiplication (MVM) output, where the aggregated MSB and LSB results are summed with the bias and compared against a calibrated threshold. This threshold determines whether the neuron will fire, resulting in a 1-bit value. This mechanism allows 8-bit inputs to be converted to 1-bit outputs, making the following layers function as pure SNN components, processing data as spike trains rather than multi-bit values.

II. Completed Tasks

II.1 TTFS SNN Attempt

A TTFS encoded design was first developed. It preprocesses 42000 training files through 33 simulation time steps ($T_{sim} = 33$) for 60 epochs. $T_{sim} = 33$ was used as an example to scale down the original files, however, ideally T_{sim} would be equal to the width of the spectrogram which represents the time dimension. It creates a "stack" of 33 different spectrograms showing how the temporal features of the audio file evolve over the timeline of the actual audio file. This can be thought of as a topographic map looking down where the first time step shows the loudest sound where it is most bright, and it gradually reveals the quieter, lower-intensity spectral components over the simulation time clock. The depth at which a spike appears becomes a direct proxy for its original decibel level. The network doesn't actually see a bright or dim pixel, but rather a fast or slow event. This effectively transforms a dense analog image into a sparse, binary 3D structure where information is encoded by not how much voltage is applied, but precisely when the first pulse arrives. While a standard convolutional neural network (CNN) processes an entire spectrogram image of the audio file in a single forward pass ($T_{sim} = 1$), the TTFS temporal-encoded SNN architecture requires computing this pass 33 separate times for a single audio file sample for the ideal temporal precision required as part of hardware constraints. The system is effectively performing approximately 83 million full forward and backward passes (derived from 42000 files $\times$ 33 steps $\times$ 60 epochs).

An average of 1.59 iterations/second per epoch was observed, equating to 14 hours to finish a full training session. This created a bottleneck in the training simulation where the hardware is forced to behave like a network that is 33 times deeper than its physical layer count, leading to an unsustainable development cycle. The learning process is further slowed by the inherent difficulty of backpropagating through time in a SNN scheme. TTFS encoding is inherently sparse and discrete, where a neuron fires exactly once or not at all. This sparsity often leads to gradient starvation. If a neuron fails to fire during a training step, its gradient becomes zero, leaving the optimizer with minimal information on how to update the weights. Facing these issues, a decision was made to pivot towards a rate-coded direct SNN training approach. In rate coding, information is represented by the average firing frequency of a neuron over a simulation window, which allows the surrogate gradient to mimic the continuous activation functions of traditional artificial neural networks (ANN), such as ReLU. This continuity ensures that small changes in synaptic weights result in incremental changes in the firing rate, providing a reliable slope for the optimizer to follow. Figure 1 below shows the accuracies of this TTFS SNN. While training accuracy steadily increases and loss decreases, validation accuracy remains stagnant around 22.7%, indicating severe overfitting. Furthermore, the linearly compounding simulation time reaches 11.43 hours, exposing the severe computational bottleneck of the TTFS temporal-encoding framework.

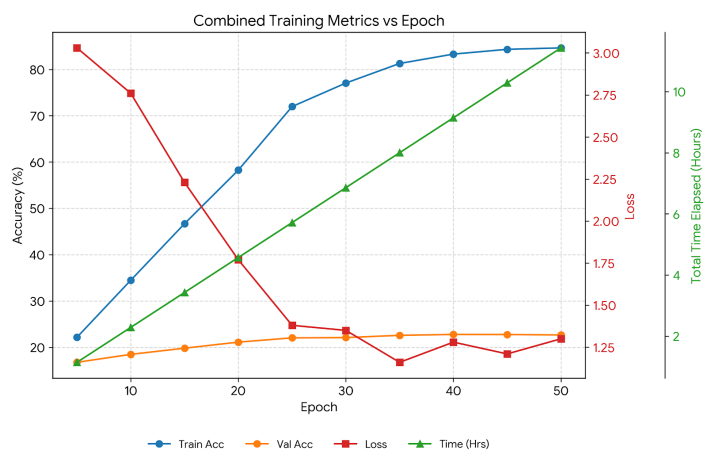


Figure 1: Combined training metrics for the TTFS SNN Attempt over 50 epochs.

II.2 Rate Coding SNN Attempt

The rate coding SNN was a convolution SNN built on Spiking Jelly's activation-based API. The idea was to take the log-normalized mel spectrogram, treat pixel values as firing probabilities, and let the network learn from the spike counts averaged over 201 timesteps. Three convolution blocks with increasing dilation and different LIF time constants ($tau = 2, 4, 8$) were followed by a fully connected layer down to

35 classes. Training utilized Adam optimization with a learning curve of 1×10^{-2}, cosine annealing down to 1×10^{-5}, a batch size of 64, and a sigmoid gradient ($alpha = 10$) for backpropagation through the non-differentiable spike functions for a total of 70 epochs. Before testing the accuracy, there was an issue with training loss showing a NaN value on the first. This issue was initially difficult to pinpoint as every training run took 20 to 30 minutes to set up. The culprit was the audio file path that the program was pointing to had corrupt files in the dataset that produced zero-variance mel spectrograms. When those hit the `log1p(mel / max)` normalization step, the division by zero propagated NaN through everything downstream. The fix was a `torch.nan_to_num()` call before normalization. Once the NaN issue was resolved, the best validation accuracy was 76.90% at epoch 66. Test accuracy came out to 77.07% with a macro F1 of 0.77. While the first 10 epochs were able to progress from 46% to 70% train accuracy, the next 60 epochs only bought a 9% increase on training and about a 4% increase on validation. By epoch 25, the validation curve was basically flat. Another issue that was faced was Kaggle's 12 hour GPU session limit. When running the neural networks, it was best decided to use a cloud-based neural network to save cost for needing a good GPU, resulting in the use of Kaggle Notebook. Unfortunately, with the design of this SNN, a single 70-epoch run took about 11.6 hours with each epoch taking about 580 seconds. This led to difficulties when trying to tune hyperparameters, as the effects of the changes would not be noticeable until enough hours were put through running it. Additionally, cloud-based notebooks are prone to crash if ran for a long time, leading to wasted experiments if it had happen. While checkpoint-and-resume logic was added to ensure not all progress was wasted, the slow feedback loop made it very hard to explore the design space. It may possible there's a rate coding configuration that breaks 80%, but finding it under these constraints would have taken months of trial and error. Figure 2 shows the accuracies of the native rate coding SNN.

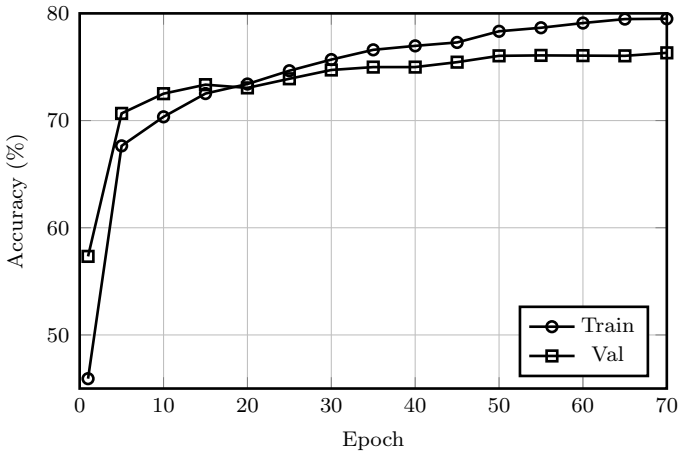


Figure 2: Native Rate Coding SNN's Training and Validation Accuracy over Epochs

II.3 ANN2SNN

Facing these constraints and time limitations, an ANN2SNN conversion approach was adopted. Since SNNs are extensions of ANNs, an ANN can first be trained and then converted into an SNN using the SpikingJelly framework. The current implementation of SpikingJelly only supports rate coding conversion, thus, a CNN was selected to process input data as 2D images.

To standardize the inputs for the CNN, raw audio waveforms were preprocessed into Mel-spectrograms. The audio was sampled at 16kHz with a 64-band Mel-scale filter and a hop length of 512ms. It should be noted that all audio waveforms were guaranteed to be 10 seconds long. This results in each spectrogram having a height of 64 and width of 313.

In particular, a ResNet architecture was employed, which incorporates residual connections to improve training stability. In a ResNet, the output is given by $x = F(x) + x$, where $F(x)$ represents the transformation learned through the convolution layers, and x as the identity mapping of the input. This architecture mitigates a vanishing gradient problem often found in deep CNNs [1]. After extensive experimentation, a ResNet-10 configuration was selected, consisting of an initial convolutional layer, three residual blocks (each containing two convolutional layers), three shortcut connections, and a fully connected layer. This configuration, with 16 initial channels, provided a favorable trade-off between feasibility and accuracy. Table 1 below shows a comparison of the initial channels and layers that have been tested. Note that these quantized SNN accuracies are evaluated at 8-bits. 8-bits was selected prematurely as prior ReRAM-based neuron designs [2] indicate that voltage-mode neurons supports a maximum output resolution of 8 bits. Parameters include both the weights and biases.

Training utilized a batch size of 16 and was capped at 45 epochs, employing an early stopping mechanism with a patience of 5 epochs to prevent overfitting. Figure 3 showcases the resulting training and validation accuracies and loss over these epochs.

Table 1: Quantized SNN Performance from Configurations

Init. Ch.	Parameters	Res. Blocks	Accuracy
8	5440	2	62.80%
8	20432	3	79.10%
8	79088	4	82.23%
16	20352	2	74.93%
16	79008	3	83.28%
16	311008	4	84.30%
32	78592	2	79.98%
32	310592	3	84.32%
32	1233344	4	86.04%

To determine if the appropriate quantization precision was actually 8 bits, 4-bit, 6-bit, and 8-bit settings were evaluated, with the optimal bit-width selected based on quantization accuracy. Among these, 8-bit quantization yielded the best performance. A comparison of the evaluated configurations is shown in Table 2. It should be noted that during training, the quantized spectrogram inputs were temporarily cast back to 32-bit floating-point representations for backpropagation.

Following training, convolution and batch normalization layers were fused to simplify the network structure for conver-

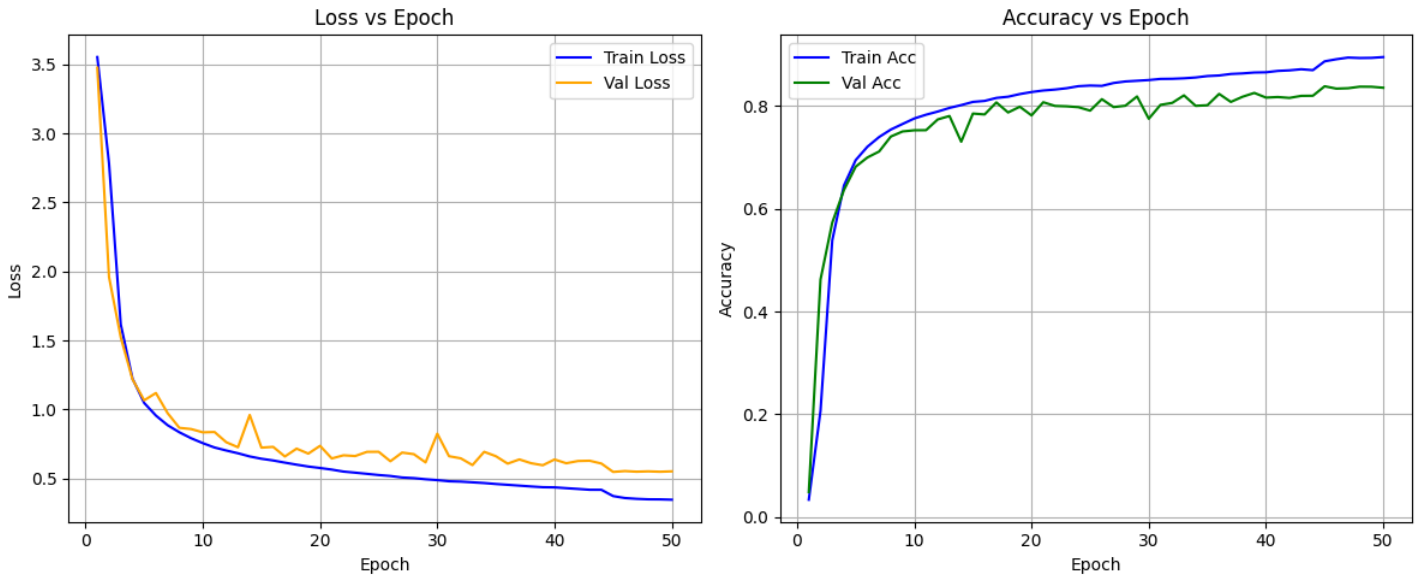


Figure 3: Training and Validation Loss and Accuracy versus Epoch

Table 2: Bit Comparison for Quantization

Bit Resolution	Accuracy
4	14.38%
6	82.30%
8	83.28%

sion. SpikingJelly was then called to perform the ANN2SNN conversion. It does this by replacing all ReLU activation functions with Integrate-and-Fire (IF) neurons. An inference window of 200 timesteps was selected to provide a sufficiently long temporal window to accurately approximate continuous ReLU activations through discrete spike rates. SpikingJelly does not support Leaky Integrate-and-Fire (LIF) neurons, necessitating the use of IF neurons. While this reduces biological realism, it simplifies circuit implementation and improves power efficiency by eliminating the need for charge leakage mechanisms. To properly configure neuron firing thresholds (V_{th}), a calibration step was performed by passing 50 spectrogram samples through the trained ANN and recording the maximum activation values at each ReLU layer. The maxes were then used by SpikingJelly to set appropriate firing thresholds for the corresponding IF neurons. As previously mentioned, the converted SNN was then evaluated to verify its quantized accuracy from the selected bit precision.

Finally, the quantized weights were mapped to conductance values for implementation on a ReRAM crossbar array. A range of 1 µS to 40 µS was used as that was the range that been chosen for CNNs in previous studies [2]. Conductance state pairs were then assigned using resistance values of 300 kΩ and 10 kΩ based on relative weight magnitudes. It should be noted that computationally, the layout is mapped vertically, thus, the weight matrices need to be transposed prior to export. The final deliverables consists of three things: synaptic weights, biases, and firing thresholds.

II.4 Core Layout

The dimensions of each layer for ResNet-10 are summarized in Table 3. Two important considerations should be noted regarding the reported totals. First, although the network contains 78,672 weights, this number must be doubled for hardware implementation. Computational model supports signed weights but physical hardware does not and therefore, each weight needs to be represented using a differential pair (G^+ and G^-). Thus, the actual weight count needs to be doubled to 157,344, requiring at least three (256×256) cores to fully map out. In particular, the number of rows for each layer must be doubled to account for differential weight representation, this is known as differential row mapping. Physically, this means that each differential pair (G^+ and G^-) is vertically stacked within the crossbar, eliminating the need for additional processing to compute the difference which is the case for differential column style mapping, thus, improving overall power efficiency. Figure 4 shows the proposed layout. Note that there is a gap between rows 144 to 160 to ensure that a 32-decoder block can work.

Second, Layer 1 does not include a shortcut entry in the table. Unlike Layers 2 and 3, the input and output dimensions of Layer 1 remain the same at 16 channels. This eliminates the need for a dedicated ReRAM crossbar for the shortcut path. Instead, the shortcut can be implemented as an identity connection to the output.

III. Remaining Tasks

The remaining work for this project consists of completing the schematic-level implementation, layout at the physical-level, and post-layout validation of the TNSA architecture. The next step involves transitioning from block-level design to transistor-level implementation using the SkyWater 130 nm PDK in Cadence Virtuoso. This includes developing schematics for the array cores and neuron circuitry, followed by functional verification to ensure correct input and output behav-

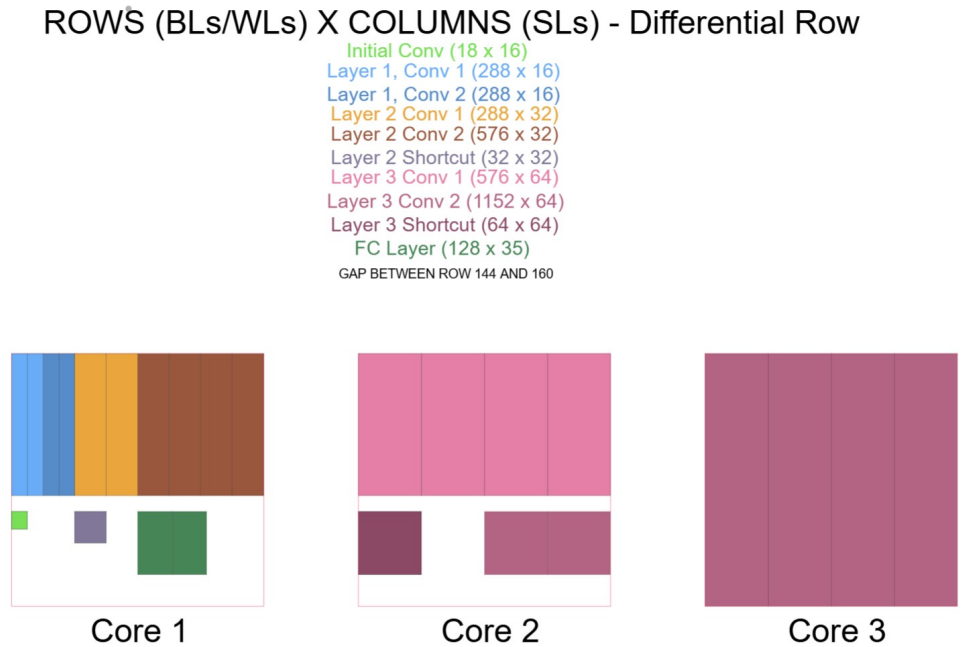


Figure 4: 3 (256 × 256) Core Layout

Table 3: SNN Weight Array Sizes (16 Initial Channels, 3 Residual Blocks)

Layer	**Dimensions**
Initial Convolution	(9 × 16)
Layer 1 Convolution 1	(144 × 16)
Layer 1 Convolution 2	(144 × 16)
Layer 2 Convolution 1	(144 × 32)
Layer 2 Convolution 2	(288 × 32)
Layer 2 Shortcut	(16 × 32)
Layer 3 Convolution 1	(288 × 64)
Layer 3 Convolution 2	(576 × 64)
Layer 3 Shortcut	(32 × 64)
Fully Connected	(64 × 35)
Total Weights	78,672
Biases	336
Total Parameters	79,008

ior. After schematic validation, full custom layout will be performed, including careful placement of differential weight cells and routing of array interconnects. The design will then need to pass Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification, followed by post-layout power and area analysis, and finally generation of the GDSII file for final tape-out delivery.

III.1 Schematic Design

Transistor-level schematics for the three array cores will be developed using Cadence Virtuoso with the SkyWater 130 nm PDK. The schematics include the ReRAM arrays consisting of the 1 NMOS transistor and 1 resistor (1T1R) cells. Each core will be implemented using differential cells (G^+ and G^-). The schematic design will also include the required bit-line (BL), word-line (WL), and source-line (SL) interconnects to support voltage-mode weighted summation and accumulation within the array.

III.2 Voltage-Mode Neuron Design

A voltage-mode neuron circuit will be implemented to sense the array line outputs and convert the analog results into a digital representation. The neuron will include a sensing amplifier that buffers and amplifies the small voltage developed on the selected array line before sampling. Sampling and integration capacitors, along with switching logic, will be used to accumulate charge across multiple cycles. This circuit captures the settled voltage from the array output lines and performs integration prior to analog-to-digital conversion.

III.3 Physical-Level Design

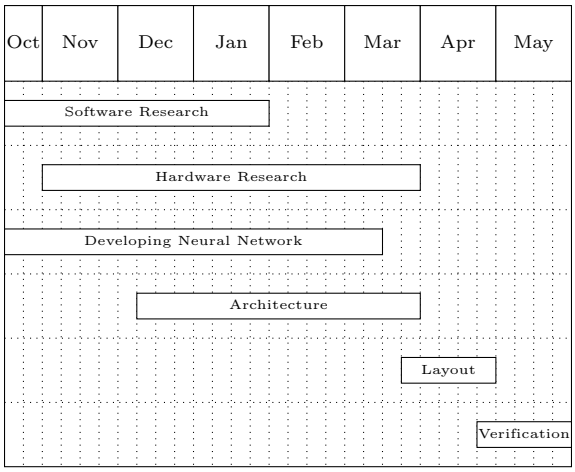
The physical layout of the three array cores and neuron circuits will be implemented, with careful placement to minimize parasitic effects and maintain signal integrity. The completed layout will then undergo DRC to ensure all layout geometries adhere to the design rules specified by the process design kit, followed by LVS verification to confirm consistency between the layout and schematic before proceeding to post-layout analysis.

III.4 Post-Layout Analysis and GDSII Generation

Post-layout simulations and measurements will be conducted to evaluate final power consumption and overall design area, with parasitics included to account for interconnect resistance and parasitic capacitance in layout. These simulations will provide more accurate estimates of total power usage and size of the cores. After completing post-layout verification and analysis, the layout will be exported in GDSII file format for tape-out.

IV. Gantt Chart

A Gantt chart illustrating the updated timeline for this project.



V. Appendix A: Advisor Meetings

- **02/02/2026**: Discussed about reducing the current SNN's neurons. PDK has not been aquired yet.
- **02/09/2026**: Clarified what a neuron is. Experimented with different channel numbers and layers. Implemented an 8-bit quantization for the spectrogram inputs. Try to create a direct SNN if possible. Chip architecture should be done by next meeting.
- **02/11/2026**: Finalized the channel number and layers to be 16 and 3 respectively.
- **02/18/2026**: Going with a three core layout. Need to design the routing for ease.
- **02/25/2026**: Figure out what exactly to draw the cell. Make slides summarizing the project progress.
- **03/02/2026**: Explain how the conversion from ANN to SNN happens with SpikingJelly. Start thinking about how to explain the data moving through the convolution layers.
- **03/09/2026**: Clarify on the time cycles for the SNN. When there is enough data gathered, it should be sent immediately.
- **03/23/2026**: Fixed technical difficulties regarding the PDK and Cadence Virtuoso. Apply resistance values of $300\,\mathrm{k}\Omega$ and $10\,\mathrm{k}\Omega$ to the conductances for mapping.

VI. Appendix B: Internal Meetings

- **02/07/2026**: Shifted to only focusing on the ANN2SNN conversion.
- **02/14/2026**: Discussed the architecture and how to make the core layout.
- **02/20/2026**: Finalized an initial three core layout.
- **02/28/2026**: Developed slides showcasing the current position of the project.
- **03/05/2026**: Changed the core layout from differential column to differential row.
- **03/15/2026**: Looked over the PDK in Cadence Virtuoso.
- **03/24/2026**: Running experiments with a fully quantized network and different bit quantization.

References

[1] Lokesh Borawar and Ravinder Kaur. Resnet: Solving vanishing gradient in deep networks. In *Proceedings of International Conference on Recent Trends in Computing: ICRTC 2022*, pages 235–247. Springer, 2023.

[2] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, et al. A compute-in-memory chip based on resistive random-access memory. *Nature*, 608(7923):504–512, 2022.